price, cost, length

r = revenue

1) RodCut (p, c, n)

```
let r[0..n]
let k[0..n]
```

] → new Arrays

```
r[0] = 0, k[0] = 0

for (i = 1, i ≤ n)
    m = p[i]
    for (j = 1, j ≤ i-1)
    m = max(m, p[i] + r[i-j] - c)
    If (p[j] + r[i-j] - c > m)
        k[i] = j
    r[i] = m
    return (r, k)
```

This is the bottom-up cut rod algorithm which figure out optimum order to fill the solution array

The running time for this is $O(n^2)$ time

cost

→ rod of length

){

Array

-1;

]+ a[x-y] = c)

algorithm which

the solution array.

$O(n^2)$ time

(2) Let $X[0..n-1]$ → input sequence of length → n

Let $L(0,n-1)$ → length of longest palindromic sequence

When last character == first character,

$L(0,n-1) = L(1,n-2)+2$

else

$L(0,n-1) = \max(L(1,n-1), L(0,n-2))$

Base case → for $i=0$ to n, i++, $L[i][i]=1$

Dynamic Programming recursive step

↳ for $cl=2$ to n, cl++

for $i=0$ to $n-cl+1$, cl++

$k = i-1+cl$

input string that is given already

2D Matrix

if($cl==2$ && $st[k]==st[i]$ then

$L[i][k]=2$

else if ($st[k]==st[i]$ then

$L[i][k] = L[i+1][k-1]+2$

else

$L[i][k] = \max(L[i][k-1], L[i+1][k]$

Time Complexity is $O(n^2)$

This is basically use recurssive algorithm. this has overlapping subproblem property which means LPS has both properties of dynamic programming

3 Dynamic Programming

$P(j) \rightarrow$ max profit that can be opened from

$m1 \quad m2 \quad m3 \sim\sim\sim \ldots mj$

Base case of this Dynamic programming algorithm is

$P(j) = Max(P1 \sim Pj)$ when $k \geq j \geq 1$.

Algorithm start

```
BaseMax = -∞
for (i=1, i ≤ k){
    if (Pi > BaseMax){
        BaseMax = Pi;
        P[i] = BaseMax;
    }
}
for (i=k+1; i ≤ n){
    if(P[i-1] ≤ P[i-k]+P[i]){
        P[i] = P[i]+P[i-k]
    } else{
        P[i] = P[i-1]
return P[n]
```

Argue and Analyze running time

for the step 1 ~ 11, which is Inductive step, that is $n \geq i > k$ in location i, when we open restaurant that is maximum is $Pi + P[i-k]$ because other restaurant assumed to be place over $i-k$. We can't place the restaurant when maximum profit is $P[i-1]$. So the algorithm is accurate. Thus, the run time for this algorithm is $O(n)$

4 Let's assume we have a sequence of 3, 10, 1, 1

(a) when first player is greedy and take first card that is 3. After that, when next player take card value that is 10, player who take card secondarly win the game. So, the natural greedy strategy is not guaranteed to be a winning strategy for the first player.

(b) In order to get first player's optimal result, we need to use dynamic programming.

Subproblem ⇒ $A(i, j)$ has the difference between largest total that the first player contain and certain score of second player on sequence $s_i, s_{i+1}, s_{i+2} \dots s_j$.

Algorithm ⇒ $A(i, j) = \max\{s_i - A(i+1, j), s_j - A(i, j-1)\}$

Optimaltotal (A, n)

$\forall i \in S$

Init $A(i, i) = s_i$

$\forall i \leq j$

$A(i, j) = \max\{s_i - A(i+1, j), s_j - A(i, j-1)\}$

return $A(1, j)$

when the user choose first card, In case he will take score $-A(i+1, j)$ and s_i or take $-A(i, j-1)$ and s_j on card that is remaining.

The running time of subproblem and M is $O(n^2)$ and $O(1)$ for update

5 Add [X
every ele

Small set (
Sort se
Result

for (i

Res

elem

retu

Sorted
which me
element
set fo

Time (

Sort

3, 10, 1, 1
first card
ter take card
d secondarly
Strategy
strategy

al result,

tween largest
certain score
2 ... 5j.

(i, j+1)}

will take
+1) and

n^2) and O(1)

5 Add $[x_i, x_{i+1}]$ to the consequence set and remove every elements in set that has $[x_i, x_{i+1}]$

```
Small set(X)
  Sort set X
  Result = null → empty set

  for (xi = 0, xi ≤ X.length)
      Result ± [xi, xi+1] and get rid of every
      elements in [xi, xi+1]

      return Result
```

Sorted element → remove element $[x_i, x_{i+1}]$ which means optimal. because we confirmed that required element are removed when it's in $[x_i, x_{i+1}]$, add only new set for each loop iteration.

Time Complexity ⇒ $O(n \log n)$

Sort need $O(n \log n)$ & Scan → $O(n)$ time

$O(n \log n) + O(n) = O(n \log n)$